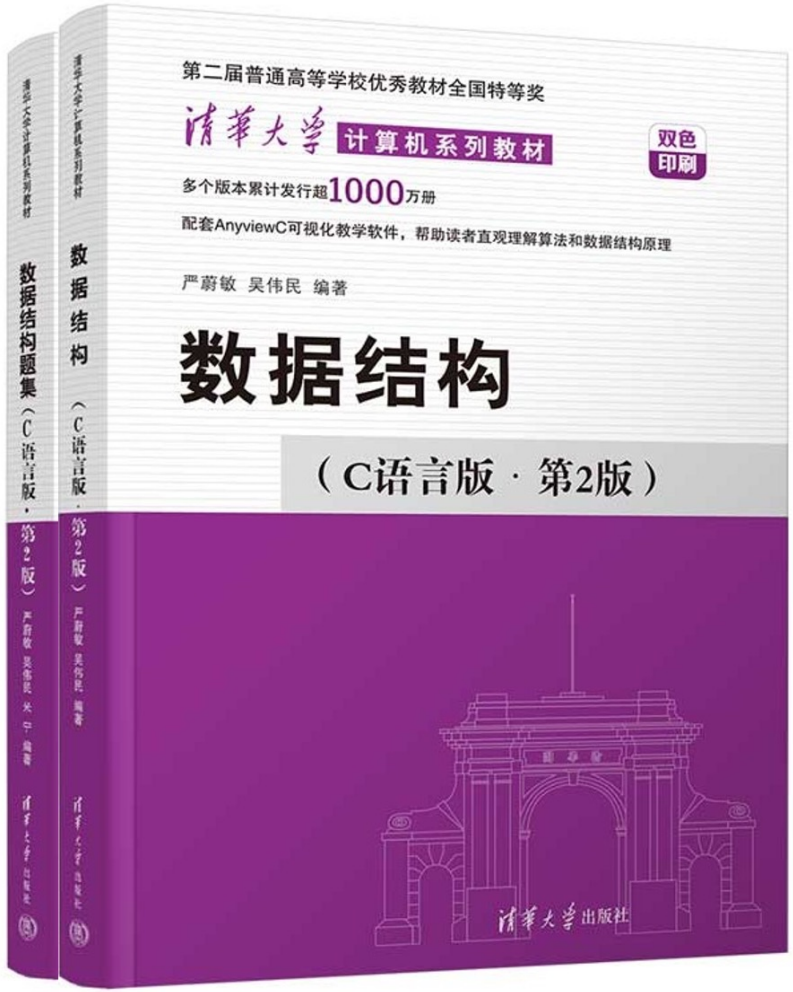


清華大学出版社
TSINGHUA UNIVERSITY PRESS

第7章

图

7.1 图的定义和术语 —— 抽象数据类型

清華大學出版社 TSINGHUA UNIVERSITY PRESS

ADT Graph {
数据对象V：V是具有相同特性的数据元素的集合，称为顶点集。
数据关系R：
R = {VR}
VR = { <v,w> | v,w∈V且P(v,w)，<v,w>
谓词P(v,w)定义了弧<v,w>的意义或信息 }
基本操作P：

MakeGraph(V, VR);
初始条件：V是图的顶点集，VR是图中弧的
操作结果：按V和VR的定义构造并返回图。

FreeGraph(G);
初始条件：图G存在。
操作结果：回收图G并返回NULL。

LocateVex(G, v);
初始条件：图G存在，v和G中顶点有相同特
操作结果：若G中存在v顶点，则返回其在

GetVex(G, v);
初始条件：图G存在，v顶点在G中。
操作结果：返回v的值。

PutVex(G, v, value);
初始条件：图G存在，v顶点在G中。
操作结果：对v赋值value。

FirstAdjvex(G, v);
初始条件：图G存在，v顶点在G中。
操作结果：返回v的第一个邻接顶点。若顶点

NextAdjvex(G, v);
初始条件：图G存在，v顶点在G中。
操作结果：返回v的下一个邻接顶点。若不存在,返则回 nullP。

Insertvex(G, v);
初始条件：图G存在，v和图中顶点有相同特征。
操作结果：在图G中增添v顶点。

Deletevex(G, v);
初始条件：图G存在，v顶点在G中。
操作结果：删除G中顶点v及其相关弧。

InsertArc(G, v, w);
初始条件：图G存在，v顶点和w顶点均在G中。
操作结果：在G中增添弧<v,w>,若G是无向的，则还增添对称弧<w,v>。

DeleteArc(G, v, w);
初始条件：图G存在，v顶点和w顶点均在G中。
操作结果：在G中删除弧<v,w>,若G是无向的，则还删除对称弧<w,v>。

DFSTrav(G, Visit());
初始条件：图G存在，visit是顶点的应用函数。
操作结果：对图进行深度优先遍历。在遍历过程中对每个顶点调用函数visit一次且仅一次。一旦visit()失败，则操作失败。

BFSTrav(G, visit());
初始条件：图G存在，visit是顶点的应用函数。
操作结果：对图进行广度优先遍历。在遍历过程中对每个顶点调用函数visit一次且仅一次。一旦visit()失败，则操作失败。

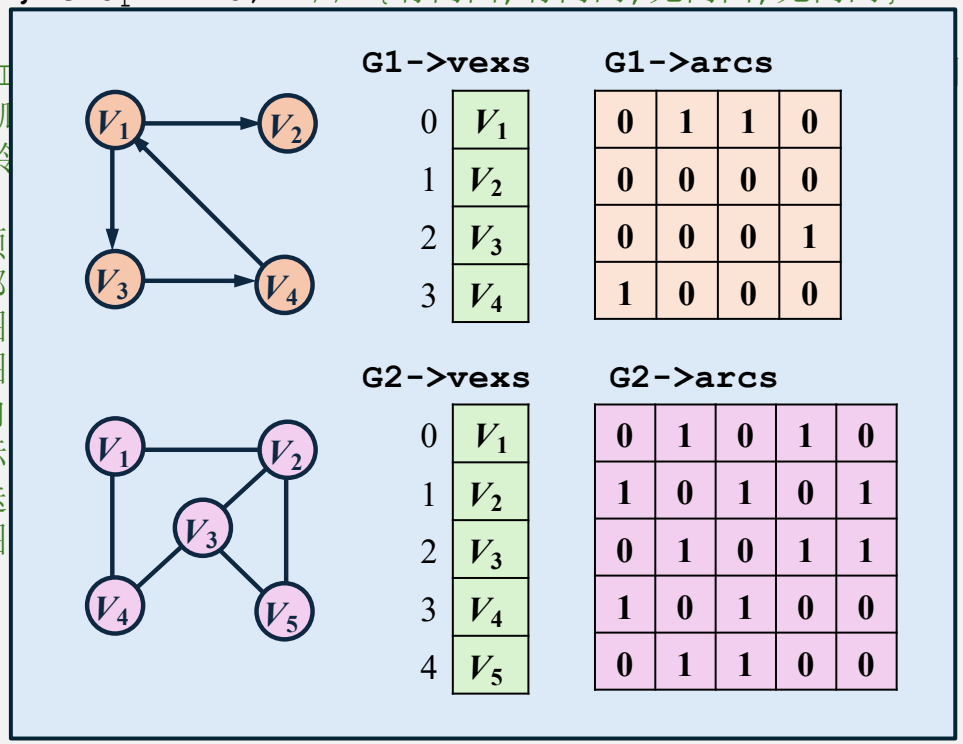
} ADT Graph

7.2 图的存储结构

7.2.1 数组（邻接矩阵）表示法

```
typedef enum {DG, DN, UDG, UDN} GraphKind;   // {有向图,有向网,无向图,无向网}
typedef struct ArcCell {
   VRType      adj;        // VRT
   InfoType    info;       // 该弧
} ArcCell,  **AdjMatrix;    // 令
typedef struct {
   VexType    *vexs;       // 顶
   AdjMatrix   arcs;       // 邻
   int         n, a;       // 图
   GraphKind   kind;       // 图
   int         count;      // 内
   int        *visited;    // 标
   int        *curs;       // 迭
} *MGraph;                 // 图
```

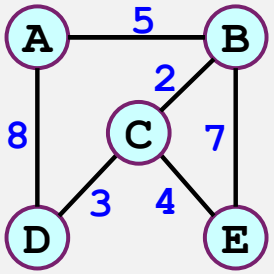
为权值类型

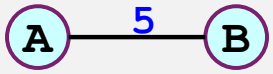


例如，图7.1的图G1和G2的
邻接矩阵存储结构如图7.8所示

算法7.1和7.2 无向网的邻接矩阵构造

清華大學出版社 TSINGHUA UNIVERSITY PRESS



算法7.1提供了构造图的数组（邻接矩阵）表示的统一接口。
参数包括要构建的图G及其种类kind，n个顶点的序列v，e条边或弧的三元组序列a。
右边的无向网，顶点为字符类型char，传递给参数v的是字符串"ABCDE"；
边的三元组由其两个顶点和权值构成，如边 

的三元组为：{A,B,5}

传递给参数a的是序列：{{A,B,5},{E,C,4},{C,D,3},{D,A,8},{B,C,2},{E,B,7}}

```
MGraph CreateGraph(GraphKind kind, VexType *V, int n, ArcInfo *A, int e) {
   // 算法7.1 采用数组（邻接矩阵）表示法，按指定种类kind构造n个顶点和e条边（弧）的图或网G。
   if(n<0 || e<0 || !V|| !A) return NULL;
   switch (kind) {
      case  DG: return CreateDG(V, n, A, e);  // 创建有向图
      case  DN: return CreateDN(V, n, A, e);  // 创建有向网
      case UDG: return CreateUDG(V, n, A, e); // 创建无向图
      case UDN: return CreateUDN(V, n, A, e); // 创建无向网，调用算法7.2
      default: return NULL;
   }
}
```

算法7.1和7.2 无向网的邻接矩阵构造

清華大學出版社 TSINGHUA UNIVERSITY PRESS

图有四种常用存储结构。第一种是数组表示法，常称为邻接矩阵。
数组表示法的主要结构是顶点向量和邻接矩阵，都需要动态分配内存。
每个顶点存储在一个结点，所有顶点结点构成顶点向量，其下标也作为顶点的序号，便于访问顶点以及在邻接矩阵中对应顶点之间的关系，即图的边或弧。顶点向量按顶点个数分配存储空间。
常用的标志向量和迭代向量也封装在类型中，和顶点向量类似，也按顶点个数分配存储空间。

```
#define INFINITY INT_MAX;   // 权值的最大值∞
typedef enum { DG, DN, UDG, UDN } GraphKind;
        // 作为有向图、有向网、无向图、无向网的种类标志
typedef VRType **AdjMatrix; // 邻接矩阵为二重指针类型，图初建时动态分配空间
        // VRType是顶点的关系类型，作为邻接矩阵矩阵的元素类型，常用整型int
typedef struct {
   VexType     *vexs;      // 顶点向量。顶点类型VexType约定是字符类型char，除非特别说明
   AdjMatrix    arcs;      // 邻接矩阵
   int          n, a;      // 顶点数与边数
   GraphKind    kind;      // 图的种类标志
   int         *visited;   // 访问标志向量，可在使用时动态分配和回收，常用于图遍历时的访问标志
   int         *curs;      // 迭代向量，支持FirstAdjVex、NextAdjVex等基本操作的实现
} MGraph;    // 邻接矩阵类型
```

算法7.1和7.2 无向网的邻接矩阵构造

清華大學出版社 TSINGHUA UNIVERSITY PRESS

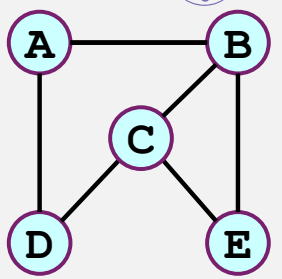
邻接矩阵定义为二重指针类型，由顶点个数作为邻接矩阵的行列数，可调用函数callocAdjMatrix为邻接矩阵动态分配存储空间。
例如，一个含有5个顶点的无向图G的邻接矩阵是一个5◊5的方阵。
可观看以下调用的执行过程演示。

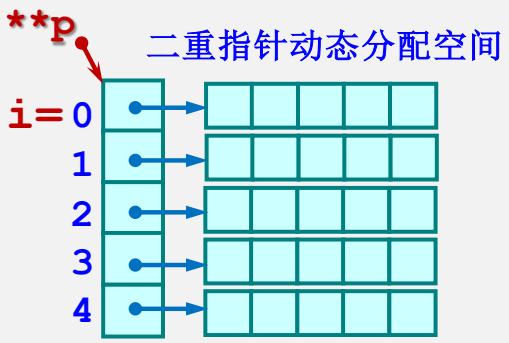
```
G->arcs = callocAdjMatrix(5);
```

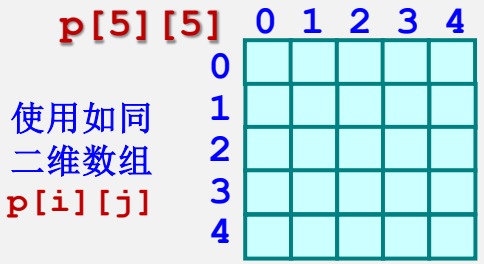
```
AdjMatrix callocAdjMatrix(int n) {
  AdjMatrix p;
  p = (ArcCell**)malloc(n*sizeof(ArcCell*));
  for (i=0; i<n; ++i)
    p[i] = (ArcCell*)calloc(n, sizeof(ArcCell));
  return p;
}
```

C语言的二维数组是静态存储分配的，但与二重指针相似，差别只是它的每行空间是首尾依次相接连续的，但访问数组元素的机制和二重指针相同，因此，我们可以把上述动态分配的邻接矩阵看作二维数组，用行列下标访问元素。

采用二重指针优点：可以按需分配存储空间。

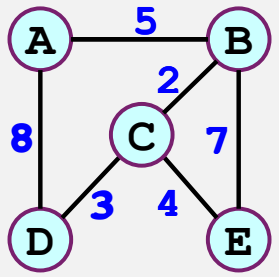




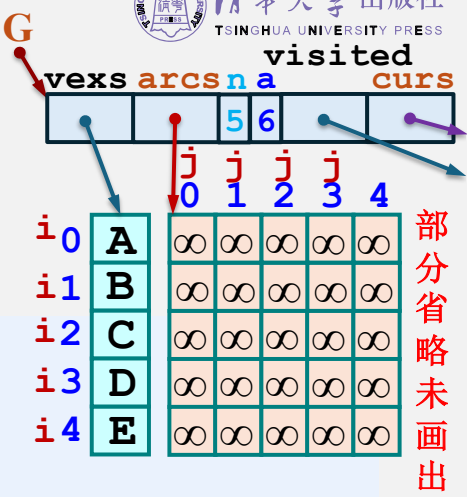


算法7.1和7.2 无向网的邻接矩阵构造

以右边的无向网为例，算法的执行过程演示



6条边{{A,B,5},{E,C,4},{C,D,3},
5个顶点{A, B, C, D, E}　{D,A,8},{B,C,2},{E,B,7}}




```
MGraph CreateUDN(VexType *V, int n, ArcInfo *A, int e) {
   // 采用数组（邻接矩阵）表示法，构造无向网。v含n个顶点，a含e条边及权值。
   MGraph G;   int i, j, k;
   if(!(G = (MGraph)malloc(sizeof(*G))))exit(OVERFLOW);
   G->n = n;  G->a = e;  G->kind = UDN;
   if(!(G.vexs = (VexType*)calloc(n, sizeof(VexType))))exit(OVERFLOW); // 分配顶点向量空间
   for (i=0; i<n; ++i) G->vexs[i] = V[i];    // 构造G的顶点向量
   G->arcs = callocAdjMatrix(n);              // 邻接矩阵动态分配空间
   if(!(G->visited = (int*)calloc(n, sizeof(int))))eixt(OVERFLOW); // 为访问标志数组分配空间
   if(!(G->curs = (int*)calloc(n, sizeof(int))))exit(OVERFLOW);  // 为迭代数组分配空间
   for (i=0; i<n; ++i)
      for (j=0; j<n; ++j) G->arcs[i][j].adj = INFINITY;  // 初始化邻接矩阵 ∞
   for (k=0; k<e; ++k) {  // 构造邻接矩阵
      i = LocateVex(G, A[k].v1);  j = LocateVex(G, A[k].v2);  // 确定两个顶点的序号
      G->arcs[i][j].adj = G->arcs[j][i].adj = A[k].weight;    // 在邻接矩阵置一对对称弧权值
   }
   return G;
}
```

可在AnyviewC可视跟踪算法过程中邻接矩阵形态变化

7.2.2 邻接表

可在AnyviewC可视跟踪算法过程中邻接表形态变化

邻接表(adjacency list)是图的一种链式存储结构。在邻接表中,对图中每个顶点建立一个单链表，第i个单链表中的结点表示依附于顶点v_i的边(对有向图是以顶点v_i为尾的 弧)。

```
//----- 图的邻接表存储表示 -----
typedef struct ArcNode {
    int       adjvex;      // 该弧所指向的顶点的位置
    ArcNode *nextarc;      // 指向下一弧结点的指针
    WeightType  weight;    // 弧权值
    InfoType    info;      // 弧的其他信息
} ArcNode, *ArcNP;         // 弧结点类型，弧结点指针类型
typedef struct {
    VexType   data;        // 顶点数据
    ArcNode  *firstarc;    // 顶点的邻接链表头指针，
                           // 指向第一条依附该顶点的弧结点
} VNode, *AdjList;         // 顶点类型，顶点动态数组指针类型
typedef struct {
    AdjList    vexs;       // 顶点动态数组
    int        n, a;       // 图的当前顶点数和弧数
    GraphKind  kind;       // 图的种类标志
    int       *visited;    // 顶点标志动态数组
    ArcNP     *curs;       // 邻接表迭代指针动态数组
} *ALGraph;                // 邻接表指针类型
```

讨论中常忽略此域

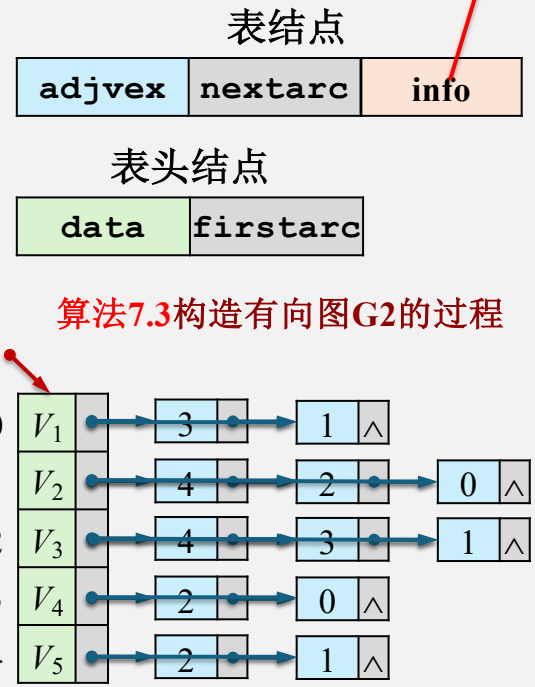


图7.10 (b) G_2的邻接表

7.2.3 十字链表

算法7.4 构建有向图的十字链表

十字链表是有向图的另一种存储结构，可看作邻接表好逆邻接表的合并集成。

有向图基本概念：

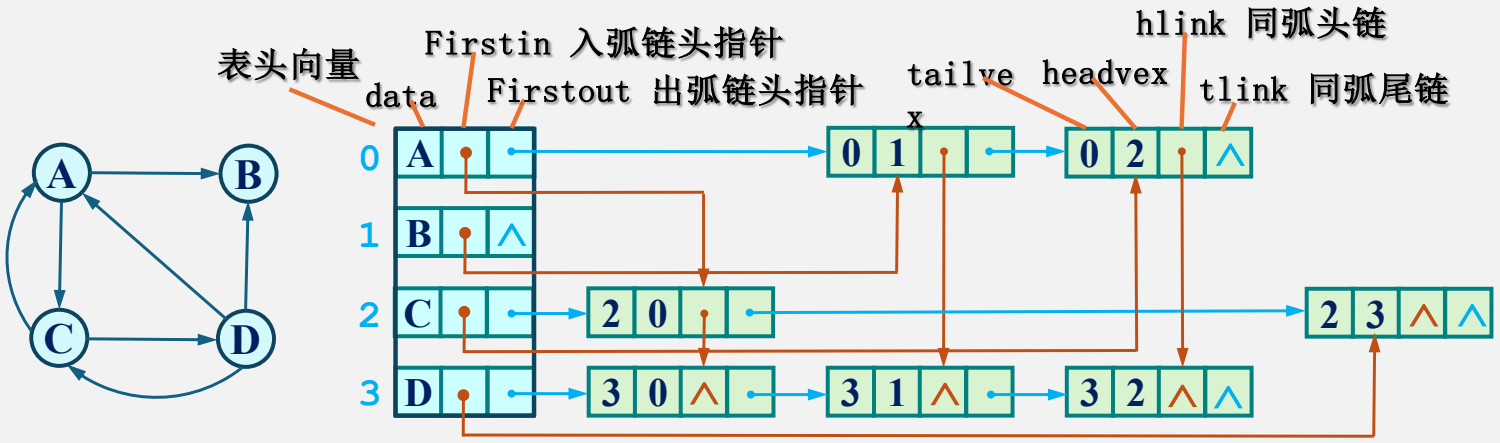
（1）弧——有向边， 逻辑上用连接顶点的带箭头线段表示 

（2）弧头和弧尾—— v为弧尾，w为弧头，该弧是v的出弧、w的入弧

图7.11是一个含4个顶点、7条弧的有向图及其十字链表。

观察每个顶点的入弧链表和出弧链表的构成。

对应关系：
入弧链表——逆邻接表
出弧链表——邻接表

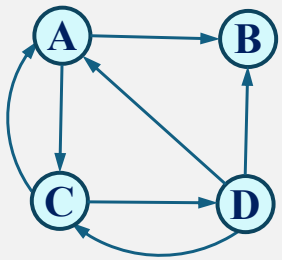


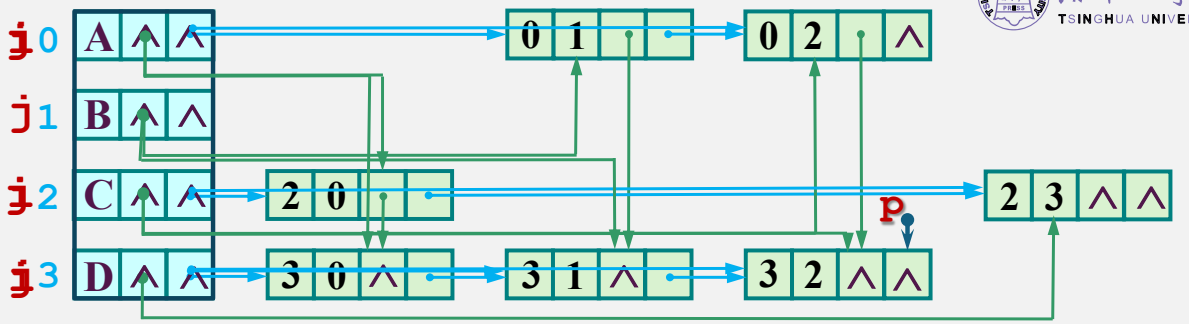
算法5.4构建稀疏矩阵的十字链表，每个非零元结点属于相应的行、列链表，按行、列号递增有序插入。

算法7.4构建有向图的十字链表，每条弧属于同弧头链和同弧尾链，但在链中不必有序，可在链头插入，较简单。

图的十字链表还可看成是邻接矩阵的链式表示，弧较稀疏时，邻接矩阵近似稀疏矩阵，十字链表是其压缩存储。

算法7.4对示例有向图G，构造十字链表过程





顶点序列{A, B, C, D}

弧（尾-头对）序列 {{D,C}, {A,C}, {D,B}, {C,D}, {D,A}, {C,A}, {A,B} }

初始化

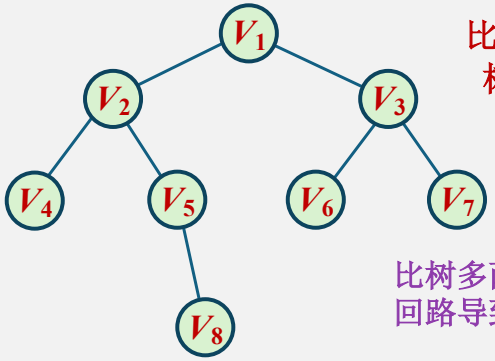
```
OLGraph MakeDG(char *v, int n, ArcInfo *ve, int e) {
  // 算法7.4 采用十字链表存储表示，构造有向图G（G.kind=DG）
  if (!(G = (ALGraph)malloc(sizeof(*G)))) exit(OVERFLOW);  // G的结构记录
  G->n=n;  G->a=e;  G->kind=DG;  G->xlist = (VexNode*)calloc(n, sizeof(VexNode));
  for (i=0; i<G->n; ++i)      // 构造表头向量
   { G->xlist[i].data = v[i];  G->xlist[i].firstin = G->xlist[i].firstout = NULL; } // 初始化
  ●●●●●● // 此略头、尾弧链迭代指针数组和访问标志数组的分配
  for (k=0; k<G->a; ++k) {    // 输入各弧并构造十字链表
    i = LocateVex(G, ve[k].v1);  j = LocateVex(G, ve[k].v2);       // 确定v1和v2在G中位置
    if (!(p = (ArcBox*)calloc(1, sizeof(ArcBox))))exit(OVERFLOW); // 分配弧结点空间
    p->tailvex = i;    p->headvex = j;      // 弧尾i，弧头j
    p->hlink = G->xlist[j].firstin;         // 插入j的入弧链（以j为弧头的弧链）
    p->tlink = G->xlist[i].firstout;        // 插入i的出弧链（以i为弧尾的弧链）
    G->xlist[j].firstin = G->xlist[i].firstout = p; // 完成在入弧和出弧链头的插入
  }
  return G;
}
```

可在AnyviewC可视跟踪算法过程中十字链表形态变化

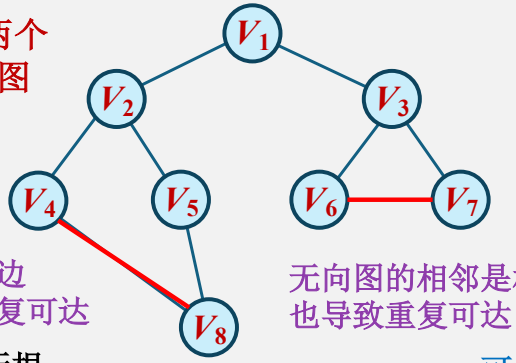
算法7.5 无向图的深度优先遍历

遍历：要求对每个顶点都访问1次且仅1次。
二叉树的递归地先序遍历确保了这一遍历要求。
图呢？无向图呢？先比较树和图的特点。
对连通图进行深度优先搜索遍历的要点：
（1）到达v，即访问，打已访问标记 “Visited”；
（2）对v的每个邻接顶点发起1次且仅1次搜索：
先取首个邻接顶点：FirstAdjVex(v)；
然后逐个邻接顶点：NextAdjVex(v)；
依次递归地深度优先搜索遍历

```
Status (* VisitFunc)(int v);          // 函数变量
void DFS(Graph G, int v) {
  // 从第v个顶点出发递归地深度优先遍历图G
  G->visited[v]=Visited;   VisitFunc(v); // 访问第v顶点
  for(w=FirstAdjVex(G,v); w>=0; w=NextAdjVex(G,v))
    if(UNVisited==G->visited[w]) DFS(G,w);
                    // 对v的尚未访问的邻接顶点w递归调用DFS
}
```



比较两个
树和图



比树多两条边
回路导致重复可达

无向图的相邻是相互的
也导致重复可达

（1）有根
（2）父子关系、有向
（3）子树是划分的、不相交
（4）可先、中、后序遍历
和层次遍历

无根 —— 可从任一顶点出发

平等相邻关系、有向或无向 —— 从所有邻接顶点发起搜索

子图可相交重叠 —— 打标记区分访问与否

深度优先搜索遍历、 —— 单兵突进搜索，有多远走多远
广度优先搜索遍历 —— 步步为营，逐渐扩大搜索范围

算法7.5 无向图的深度优先遍历

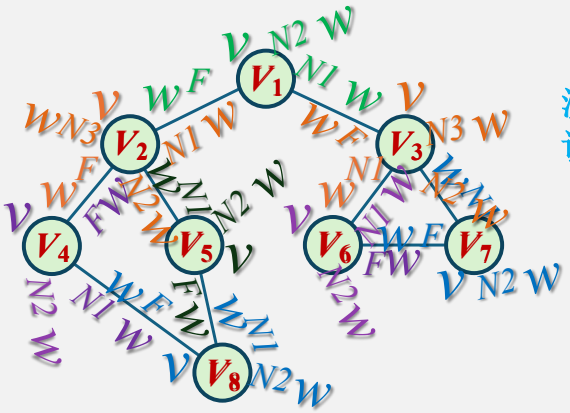
算法7.5 的DFS函数递归地深度优先搜索遍历图G的一个连通子图。

图G内设标记数组，对访问的结点打“Visited”标记，以区分顶点访问与否？

FirstAdjVex(G,v)和NextAdjVex(G,v)这对迭代操作，帮助依次获取v顶点的每个邻接顶点。

下面结合示例，讲解演示对连通的无向图图G进行递归地深度优先搜索遍历的过程。

提示：形式上算法与具体存储结构无关，但从迭代操作获得的邻接顶点的次序实际上与存储结构以及顶点的边（弧）的相对存储位置有关。



演示中用颜色代表访问标记的变化：

UNVisited V_i

Visited V_i

深度优先序列：

```
void DFS(Graph G, int v) {
  // 从第v个顶点出发递归地深度优先遍历图G
  G->visited[v]=Visited;    VisitFunc(v); // 访问第v顶点
  for(w=FirstAdjVex(G,v); w>=0; w=NextAdjVex(G,v))
    if(UNVisited==G->visited[w]) DFS(G,w);
                  // 对v的尚未访问的邻接顶点w递归调用DFS
}
```

算法7.5 无向图的深度优先遍历

在算法7.5中， DFS只遍历一个连通图。
DFSTrav在检测到顶点未被访问时，调用DFS，
实现了对任意图的深度优先搜索遍历。

```
void DFSTrav(Graph G, Status(*Visit(int v)) {  // 算法7.5
  // 深度优先遍历图G
  VisitFunc = Visit;  // 使用外部函数变量VisitFunc，使DFS省略函数参数
  for (v=0; v<G->n; ++v) G->visited[v]=Visited; // 访问标志初始化
  for (v=0; v<G->n; ++v)
    if (UNVISITED==G->visited[w])
      DFS(G, w);    // 对尚未访问顶点w递归调用DFS
}
```

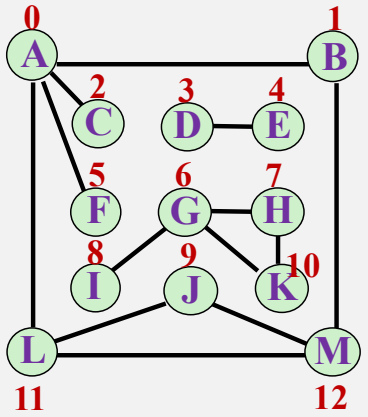


图7.3的无向图G3，共有13个顶点，分属3个连通子图。
顶点边上的数字是其序号，仍以顶点颜色表示访问与否。
算法7.5 从0号依次检测顶点，
0号A未被访问，调用DFS。
从DFS返回时，第1个连通子图已被遍历，其顶点已变为已访问的颜色。
从1号继续按序号依次检测顶点，
到3号D未被访问，再调用DFS。
从DFS返回时，第2个连通子图被遍历，顶点变色。
从4号继续按序号依次检测顶点，
到6号G未被访问，第3次调用DFS。
从DFS返回时，第3个连通子图被遍历，此时所有顶点都已变为已访问色。
但是，从7号继续检测顶点是否未被访问？
直到检测的for循环结束，完成了对图G的深度优先搜索遍历。

算法7.6 图的广度优先遍历

清華大學出版社 TSINGHUA UNIVERSITY PRESS

DeQueue　　InitQueue　　EnQueue

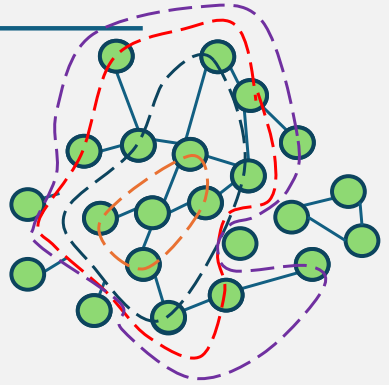
广度优先遍历采用“由近到远，步步为营”的搜索策略。

- 访问一个顶点后，依次访问它的所有邻接顶点；
- 而且，“先被访问的顶点的邻接点”先于“后被访问的顶点的邻接点”被访问；
- 如此递进，直到所有顶点都被访问。

以首访的顶点为核心，其邻接顶点围成遍历的第一圈，
由邻接关系逐步外扩，形成第二圈、第三圈、直到最外圈。

用基本操作获得邻接顶点





算法要点——如何实现这种递进式遍历？　用排队解决：

（1）访问任一指定顶点，并令其入队；
（2）让排头的顶点出队，依次对它的未访问的邻接顶点访问并入队。
（3）重复第（2）步，直到队列空为止。

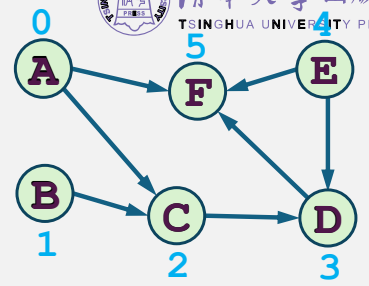
这就构成了对指定顶点所有可达顶点的广度优先遍历的算法基本架构。
例图中剩下的那些不可达的绿色顶点，可仿照算法7.5，逐个对未访问的顶点发起广度优先遍历即可。

算法7.6 图的广度优先遍历

基于排队的解题构想，形成了算法7.6的基本原型。
对一个v顶点，如果它未访问，就访问它，并从它出发广度优先遍历所有可达顶点。
如同深度优先遍历那样，用打标志的方法防止顶点被重复访问。
加一层外循环，可实现对图中所有顶点的广度优先遍历。

UNVISITED：0
VISITED：1

	0	1	2	3	4	5
G->visited	0	0	0	0	0	0




```
void BFSTraverse(Graph G, Status(*Visit)(int)) { // 对图G进行广度优先遍历
  int v, u, w;
  for (v=0; v<G->n; v++) G->visited[v] = UNVisited;  // 初始化标志数组
  Q = InitQueue();          // 初始化队列Q
  for (v=0; v<G->n; v++) // 依次检查所有顶点
    if (UNVisited==G->visited[v]) {  // 判断顶点v是否未访问
      G->visited[v] = Visited;  Visit(v);
      EnQueue(Q, v);  // 若未访问过i顶点，则访问它，并入队列
      while (!QueueEmpty(Q)) { // 若队列空，则while结束
        u = DeQueue(Q);          // 队头元素出队列到u,
        for (w=FirstAdjVex(G,u); w>=0; w=NextAdjVex(G,u)) // 扫描邻接顶点
          if (UNVISITED==G->visited[w]) {  // 若w为u上未访问过的邻接顶点
            G->visited[w] = Visited;  Visit(w);
            EnQueue(Q, w);
          }
      }
    }
  FreeQueue(Q);
}
```

Q

A C F D B E

广度优先序列

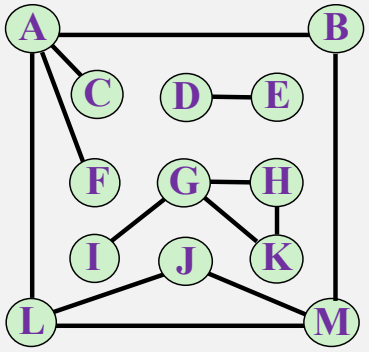
上面这个有向图，作为算法7.6遍历过程的示例。

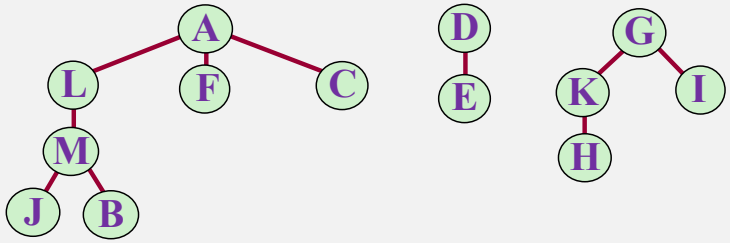
思考题：

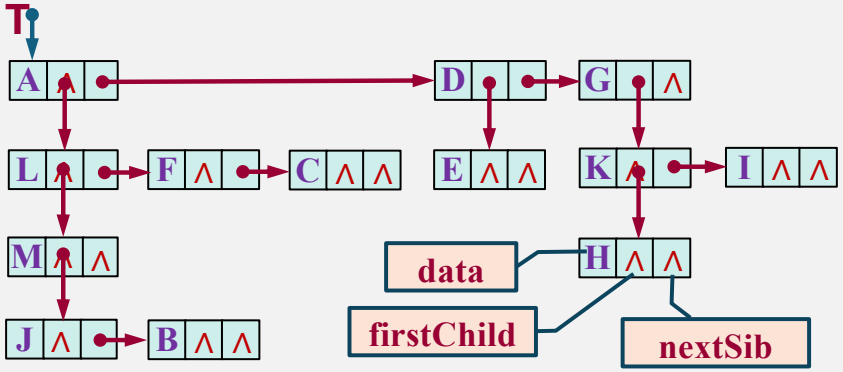
（1）为什么没用递归算法实现广度优先遍历？能不能？

（2）如果把辅助队列换成栈，还是图的遍历算法吗？如果是，那属于一种什么搜索策略呢？

清華大學出版社
TSINGHUA UNIVERSITY PRESS







无向图的深度优先生成树

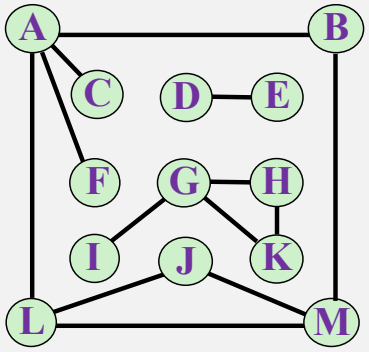
图7.3的无向图G_3，含 3 个连通分量。

无向图的深度优先生成森林不唯一，这与图的存储结构、顶点的存储位置和边的表示方式有关。

图7.15(c)给出了G_3的一个深度优先生成森林。

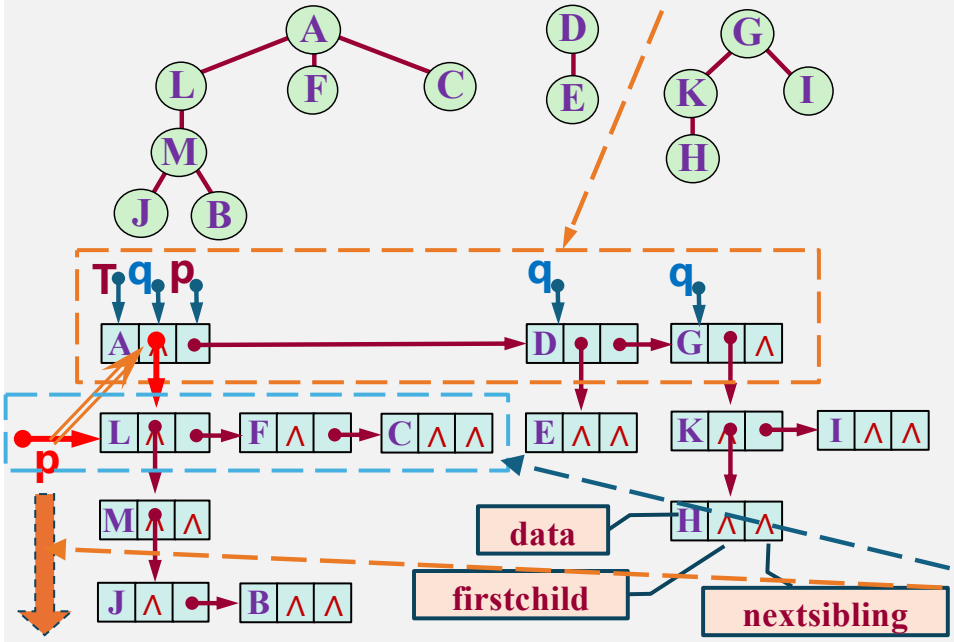
由算法7.7求得G_3的与此森林相应的存储结构，采用孩子兄弟二叉链表——CSTree类型。

在结合示例讲解演示算法7.7之前，对其两个函数各自功能和相互关系，先作一个总体的了解。



DFSForest是主控函数，以每个连通分量未访问的顶点构建对应的树根结点。
然后调用DFSTree，完成该连通分量生成树的构造。
要理解函数返回值的作用机理。

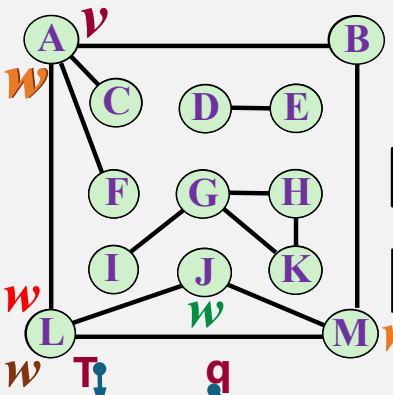
```
CSTree DFSForest(Graph G, CSTree T) {
  int v, j = 0;    CSTree p, q, T = NULL;
  for(v=0; v<G->n; ++v) G->visited[v]=UNVisited;
  for (v=0; v<G->n; ++v)
    if ( G->visited[v]==UNVisited ) {
      p = (CSTree)malloc(sizeof(CSNode));
      *p = { Getvex(G,v), NULL, NULL }; // 三个语句
      if (!T) T = p;   else q->nextSib = p;
      q = DFSTree(G, v, p);
    }
   return T;
}
```




```
CSTree DFSTree(Graph G, int v, CSTree T) {
  CSTree p, q;   G->visited[v] = Visited;    Status first = TRUE;
  for (int w=FirstAdjVex(G,v); w>=0; w=NextAdjVex(G,v))
    if (G->visited[w] ==UNVisited) {
      if(!(p = (CSTree)malloc(sizeof(CSNode))))exit(OVERFLOW);
      *p = { Getvex(G,w), NULL, NULL }; // 三个语句
      if (first) { T->firstChild = p;    first = FALSE; }
      else q->nextSib = p;
      q = DFSTree(G, w, q);  // 递归调用
    }
   return T;
}
```

for循环构建了兄弟链表。沿孩子链层层递归。

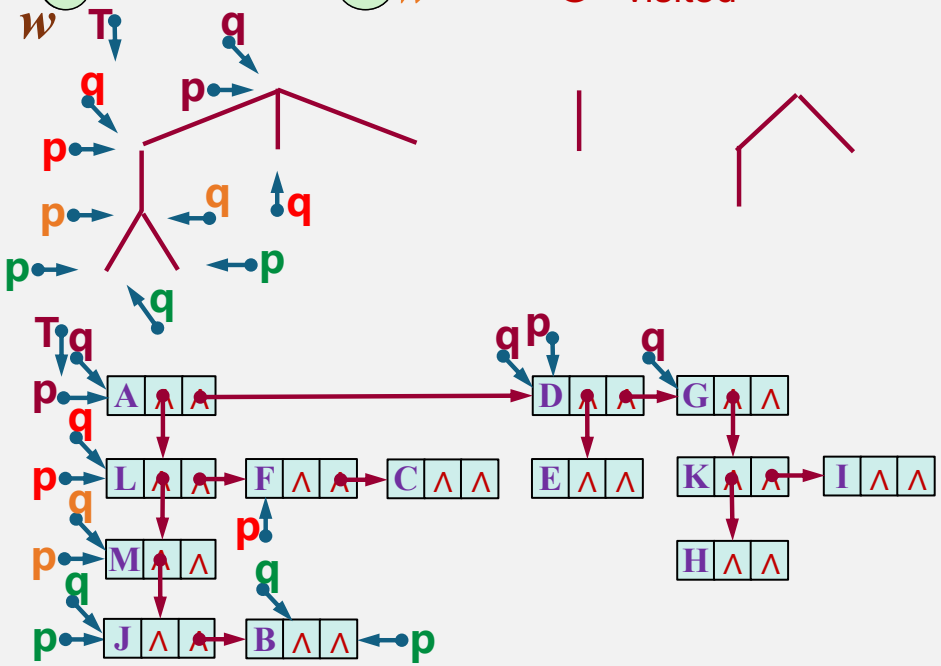
无向图深度优先生成树



顶点及其序号

A	L	M	J	B	F	C	D	G	E	K	I	H
0	1	2	3	4	5	6	7	8	9	10	11	12
1	1	1	1	1	1	1	1	1	1	1	1	1

G->visited




```
CSTree DFSForest(Graph G, CSTree T) {
  int v, j = 0;    CSTree p, q, T = NULL;
  for(v=0; v<G->n; ++v) G->visited[v]=UNVisited;
  for (v=0; v<G->n; ++v)
    if ( G->visited[v]==UNVisited ) {
      p = (CSTree)malloc(sizeof(CSNode));
      *p = { Getvex(G,v), NULL, NULL }; // 三个语句
      if (!T) T = p;   else q->nextSib = p;
      q = DFSTree(G, v, p);
    }
   return T;
}
```

生成树根结点

```
CSTree DFSTree(Graph G, int v, CSTree T) {
  CSTree p, q;   G->visited[v] = Visited;    Status first = TRUE;
  for (int w=FirstAdjVex(G,v); w>=0; w=NextAdjVex(G,v))
    if (G->visited[w] ==UNVisited) {
      if(!(p = (CSTree)malloc(sizeof(CSNode))))exit(OVERFLOW);
      *p = { Getvex(G,w), NULL, NULL }; // 三个语句
      if (first) { T->firstChild = p;    first = FALSE; }
      else q->nextSib = p;
      q = DFSTree(G, w, p); // 递归调用
    }
   return T;
}
```

-1

与存储位置相关

7.4.3 最小生成树

1. 普里姆算法

由连通带权图(连通网) $G = (V, E)$构造最小生成树T (G的连通无回路子网)，普里姆算法的基本步骤如下：

（1）若从G的顶点vi开始构造最小生成树，则T初始为只包含顶点v_i且无边的图，即$T=(\{v_i\},\{\})$;

（2）找出集合U（已入选生成树T的顶点集）和集合V-U（未入选生成树的顶点集）之间权值最小的边 (u_j, v_k)，其中$u_j \in U$, $v_k \in V$-U，简称最小边，如图9.17(a)所示，其中实线表示V-U中每个顶点和U之间的权值最小的边，粗实线表示这些最小边中的最小边(u_j,v_k)。

将此边加入到T中，并将该边仍在V-U的顶点v_k移入U，并分别更新(v_k, v_k')和(v_k, v_k'')为U-V中顶点v_k'和v_k''与U顶点之间的最小边，结果如图9.17(b)所示。

（3）重复步骤（2），直到$U=V$为止，T即为所求的最小生成树。

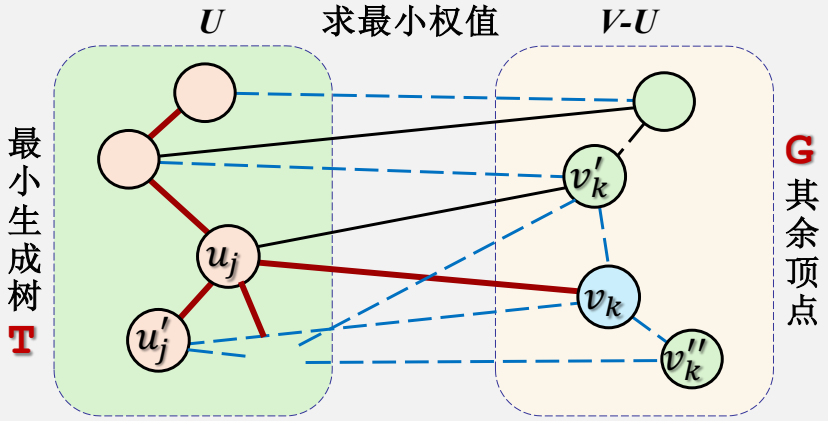


图9.17 普里姆算法的选边策略
(a) 在U和U-V之间找出最小边；
(b) 将最小边加入T（vk加入U）后, 分别更新$(vk, _{vk'})$和$(vk, _{vk''})$为U-V中顶点$_{vk}$和$_{vk''}$与U顶点之间的最小边



1. 普里姆算法（续）

普里姆算法的关键步骤是快速找出最小边。引入一维数组**closedge**：

```
typedef struct {
   int adjInd;    // U中顶点的位序
   int lowcost;   // 边的权值
} ClosedgeInfo;   // V-U的顶点和当前U之间的最小边信息类型
ClosedgeInfo  closedge[MAX_VEX_NUM]; // 全局数组
int  hpos[MAX_VEX_NUM];              // 全局数组，存储顶点在堆中位置（下标）
```

对于集合V-U中的每个顶点v_i，closedge[i]存储该顶点与集合U之间的最小边信息。在数组中按lowcost域找到的最小值即为最小边的权值。顶点0加入集合U时，将closedge[i].lowcost 置为0。

普里姆算法的步骤如下。

（1）假设从顶点v_i出发构造成最小生成树，初始子图T只包含顶点v_i：

closedge[i].lowcost = 0　　$v_i \in U$

closedge[j] = (i, w_{ij})　　$v_j \in V$-U

（2）选择其余n-1个顶点，重复执行以下步骤

1）在closedge数组中按lowcost域找到最小值closedge[k]，k∈V-U，则最小边为(k,closedge[k].adjInd)，将v_k加入到集合U，即置closedge[k].lowcost = 0，并将k顶点和边(k,closedge[k].adjInd）加入到子图T中。

2)更新closedge数组。由于集合U中新加入顶点v_k，须判断集合V-U中所有顶点和顶点v_k的边的权值是否有更小的，若有则更新closedge数组的相应值，即若w_{ij}<closedge[j].lowcost，则closedge[j]($v_j \in V$-U)=(k,w_{kj})，否则不变。

1. 普里姆算法（续）　第一步：构建辅助小顶堆H

'A'

```
MGraph MiniSpanTree_PRIM(MGraph G, VexType u) {   // 普里姆算法   算法7.9
  // 从顶点u出发构造并返回网G的最小生成树T，使用优先队列（堆）和辅助数组
  MGraph T = InitMGraph(G->vexs, G->n, UDN); // 初建仅含G的顶点但无边的最小生成树T
  Heap H = InitHeap(G->n);                   // 初始化建容量为G的顶点个数的空堆
  int i, j, k = LocateVex ( G, u );   // 确定起始顶点的序号
  for ( j=0; j<G->n; ++j ) {          // 记录从顶点集U到V－U的代价最小的边的辅助数组初始化
    if (j!=k) {                       // 初始顶点u的信息不加入辅助数组和堆
      closedge[j].adjIdx = k;    closedge[j].lowcost = G->arcs[k][j].adj;
      Insert(H, j);                   // j插入最小堆H
    }
  }
  ..... /* 构造最小生成树T */

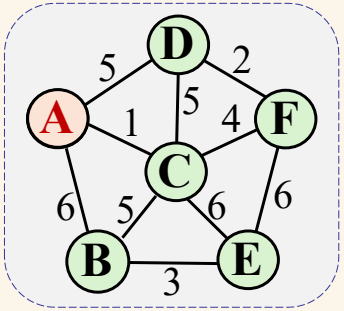
  Fr
}
```

G的邻接矩阵

0	1	2	3	4	5
A	B	C	D	E	F

$$\begin{bmatrix} \infty & 6 & 1 & 5 & \infty & \infty \\ 6 & \infty & 5 & \infty & 3 & \infty \\ 1 & 5 & \infty & 5 & 6 & 4 \\ 5 & \infty & 5 & \infty & \infty & 2 \\ \infty & 3 & 6 & \infty & \infty & 6 \\ \infty & \infty & 4 & 2 & 6 & \infty \end{bmatrix}$$

无向网G



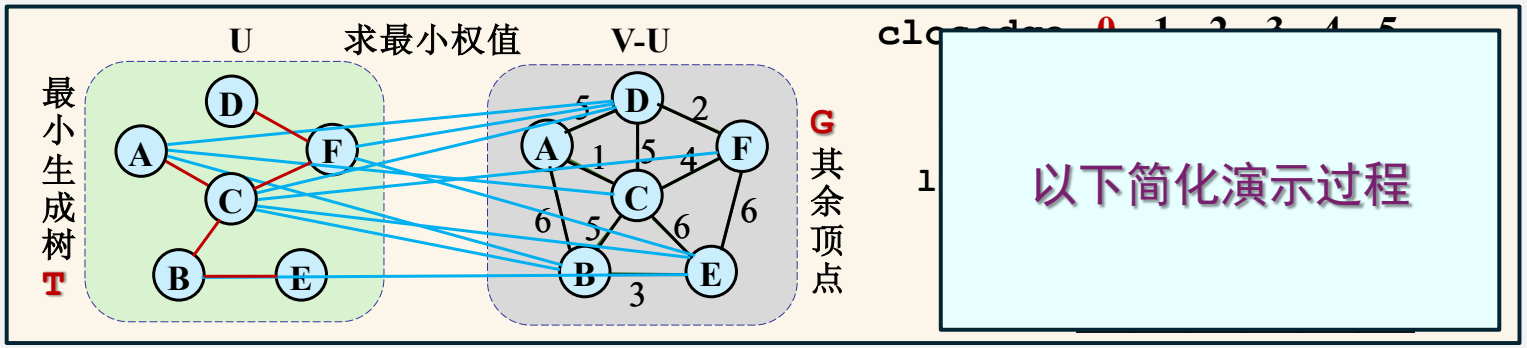
closedge

	0	1	2	3	4	5	
	A	B	C	D	E	F	k
adjIdx		0 A	0 A	0 A	0 A	0 A	
lowcost							

	0	1	2	3	4	5
H->						
hpos						

1. 普里姆算法（续）　第二步：由小顶堆H辅助构建最小生成树T

```
i=2  ..... /* 第一步：构造小顶堆H */
     MGraph T = InitMGraph(G->vexs, G->n, UDN);      // 初始顶点u加入T
     for (i=1; i<G->n; ++i) {                        // 选择其余G->vexnum-1个顶点
       k = DelTop(H);          // 删除并获取堆顶（距离最小）的顶点序号，求出T的下一个顶点：第k顶点
       if (closedge[k].lowcost == INFINITY) break;   // 若无边可选，退出循环
       T->arcs[k][closedge[k].adjIdx] = T->arcs[closedge[k].adjIdx][k]
          = G->arcs[k][closedge[k].adjIdx];          // 在T中新增一条边
       T->a++;                                       // T的边数增1
j=1    for (j=0; j<G->n; ++j)
         if (hpos[j]<=H->len)
           if (G->arcs[k][j].adj < closedge[j].lowcost) {  // 用邻接矩阵k行刷新当前最小边
             closedge[j].adjIdx = k;   closedge[j].lowcost = G->arcs[k][j].adj;
             SiftUp(H, hpos[j]);  // 类似Insert中的向上调整，与SiftDown方向相反
           }
     }
     FreeHeap(H);   if (T->a < G->n-1) printf("G is not connected!");  // 若G非联通，显示信息
     return T;
  }
```



2. 克鲁斯卡尔算法

克鲁斯卡尔算法另辟蹊径，求网$G=(V, E)$的最小生成树T（G的一个最小连通子图）的**基本思想**是

（1）最小生成树初始为只包含所有顶点且无边的非连通网$T=(V, \{\})$。

（2）从E中选取当前未被标记且权值最小的边e，标记边e，并判断边e加入T后是否产生回路，如果不产生，则将边e加入T。

（3）重复第（2）步，直到有n-1条边入选T，则算法结束，网T为所求的最小生成树。

与普里姆算法相比，克鲁斯卡尔算法的时间复杂度是$O(e\log e)$（e为网中边的数目），因而适合于求边稀疏的网的最小生成树。

按照克鲁斯卡尔算法构造连通网G的最小生成树的过程如图7.19所示。

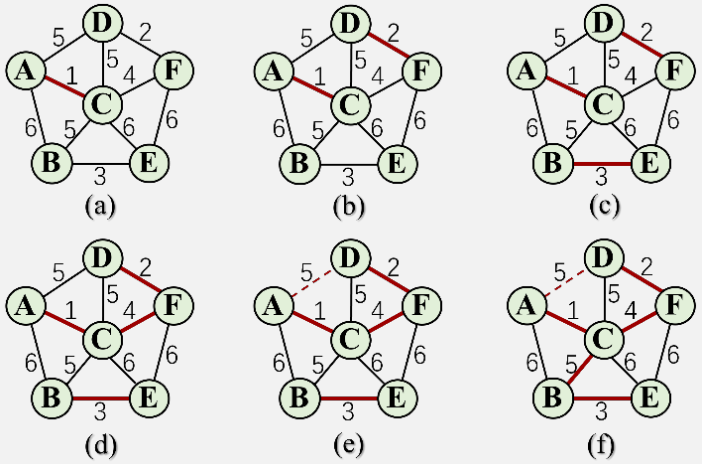


图7.19　克鲁斯卡尔算法构造最小生成树的过程示例

影响克鲁斯卡尔算法效率的有**两个要点：**

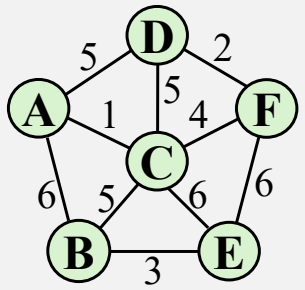
（1）**在当前未标记的边中选取权值最小的边。**可采用**小顶堆**获取权值最小的边，也可先按大小顺序将边信息存储在一维数组。取e-1条最小边的时间复杂度，二者都可由$O(e^2)$降低到$O(e\log e)$。

（2）**判断是否产生回路。**由图定义可知，在无回路的连通分量（简单连通分量）中，任意顶点之间增加一条边，必形成回路；而在两个简单连通分量之间增加一条边，则将它们合并成一个简单连通分量，不会形成回路。此问题可用6.7节的**并查集**解决，用并查集的一个子集表示一个简单连通分量的顶点集合。对于边(v, w)，若v和w同属一个子集，则加入该边必形成回路，放弃该边；否则合并v和w分属的不同子集，表示加入该边。

2. 克鲁斯卡尔算法（续）

克鲁斯卡尔算法首先构造无向网的顶点并查集。

例如图7.19(a)所示无向网的初始并查集S：



S->nodes[]

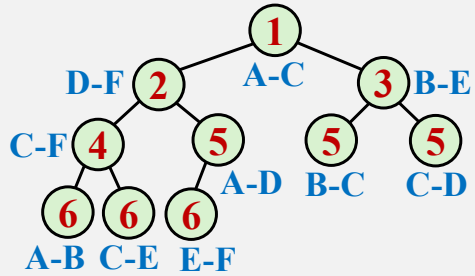
	0	1	2	3	4	5	6
parent		-1	-1	-1	-1	-1	-1

其次构造边的小顶堆H(形态与存储结构相关)：

H->elem[]

	0	1	2	3	4	5	6	7	8	9	10
v		0	3	1	2	0	1	2	0	2	4
w		2	5	4	5	2	2	3	1	4	5
weight		1	2	3	4	5	5	5	6	6	6

然后构建最小生成树T：




```
typedef struct {
  int          v, w;        // 边的关联顶点序号
  WeightType   weight;      // 边的权值
} RcdType, HElemType;       // 边信息类型，堆元素类型

Heap MakeEdgeHeap(ALGraph G) { // 对无向网的边集构造并返回小顶堆
  RcdType *edges;
  if (!(edges = (RcdType*)malloc((G->e+1)*sizeof(RcdType)))) exit(OVERFLOW);
  for (int j=1,i=0; i<G->n; i++) {    // 取G中所有边至数组edges，从下标1开始存储
    for (ArcNP p=G->vexs[i].firstArc; p!=NULL; p=p->nextArc) // 构造边集数组
      if (i<p->adjvex)    // 防止边重复（在邻接表，一条边分为两条对称弧，这里只选一条代表边）
        { edges[j].v=i;  edges[j].w=p->adjvex;  edges[j++].weight=p->weight; }
  }
  return ReformHeap(edges, G->e);      // 由含G所有边的edges数组改造并返回小顶堆
}
```

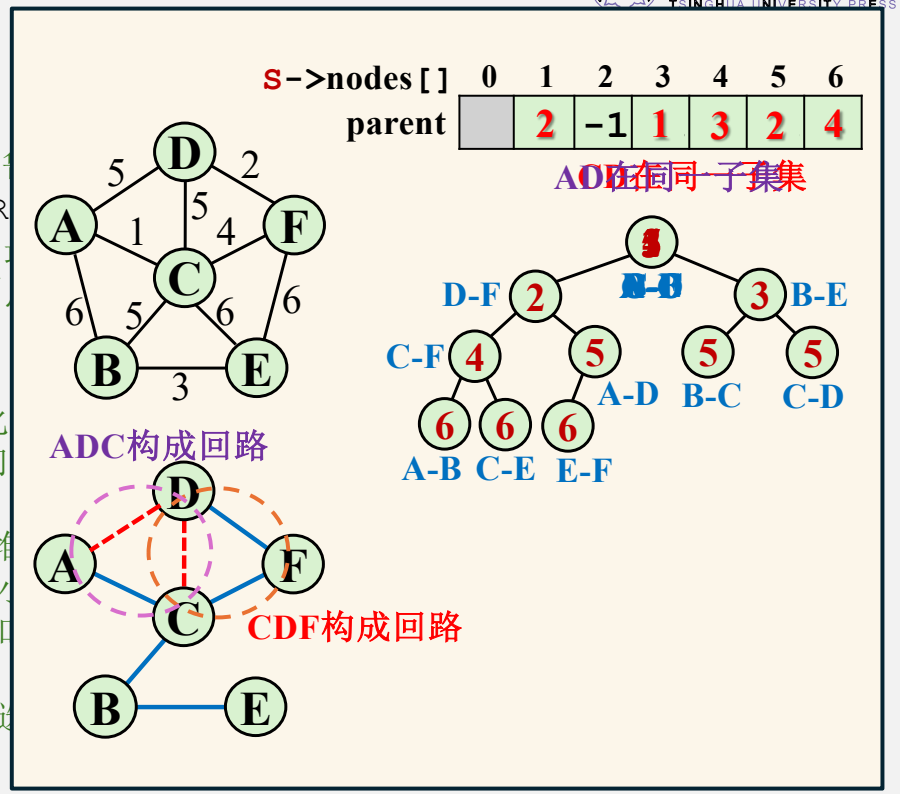
2. 克鲁斯卡尔算法（续）

如前所述，算法7.10依次构建G的顶点并查集和边小顶堆。

接着构建最小生成树T（G的一个最小连通子图）：

```
ALGraph Kruskal(ALGraph G) {  // 算法 7.10，克
  ALGraph T;  ArcNP p;  UFSet S;  Heap H;  R
  T = InitALG(G->n, UDN);              // 初始化G的
  for (int i=0; i<G->n; i++)           // 复制G的顶
    { T->vexs[i].data = G->vexs[i].data;
      T->vexs[i].firstArc = NULL; }
  S = InitUFSet_ALG(G->vexs, G->n); // 初始化
  H = MakeEdgeHeap(G);                 // 对无向
  for (int j=0; j<G->e; j++) {
    edge = DelTop(H);                  // 从小顶堆
    int v = edge.v,   w = edge.w;      // 取最小
    if (TRUE == Union(S,v+1,w+1)) {    // 若v和
      AddArc_AL(T, v, w, edge.weight);  //
      if (T->e == G->n-1) break;       // 若已选
    }
  }
  FreeHeap(H);   FreeUFSet(S);         // 回收辅助的堆和并查集
  if (T->e < G->n-1) printf("G is not connected!");  // 若G非连通网，则显示信息
  return T;                            // 返回最小生成树T
}
```

依次选最小边



7.5 最短路径

7.5.1 单个源点到其余各顶点的最短路径

迪杰斯特拉（Dijkstra）提出了一个**算法**：按路径长度递增次序求得源点v到其他所有顶点的最短路径，依次记为

$$P_1, P_2, \ldots, P_i, \ldots, P_{n-1}$$

P_i的终点记为v_i。若源点v到其他所有顶点的最短路径集

$$P = \{P_1, P_2, \ldots, P_i, \ldots, P_{n-1}\}$$

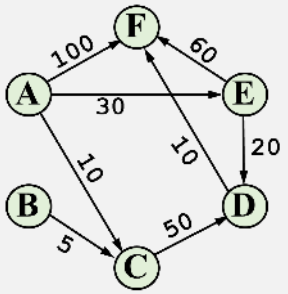
是升序集，则P具有以下两个性质：

性质1 P中长度最短的路径P_1必定只含一条弧，并且是从源点v出发的所有弧中权值最小的。

性质2 如果已求得$P_1, P_2, \ldots, P_i$，则下一条最短路径P_{i+1}或者是源点v到v_{i+1}的弧，或者是源点v经过已求得的某条最短路径P_k（$1 \le k \le i$）以及v_k到v_{i+1}的弧。

证明（反证法）：假设下一条求得的最短路径P_{i+1}不经过已求得最短路径的顶点，而经过其他顶点v_j（$i+1<j \le n-1$）到达v_{i+1}，则有等式$D(v, v_{i+1}) = D(v, v_j) + D(v_j, v_{i+1})$，其中$D(v, v_i)$表示$v$到$v_i$的路径长度，可推出$D(v, v_j) < D(v, v_{i+1})$，即源点$v$到$v_j$的路径长度更短，$P_j$是已求得的最短路径，$j$应小于$i+1$，与假设$j$大于$i+1$矛盾。

例如，图7.21(a)的有向带权图G_6，依次求得源点A到其他所有顶点的最短路径如图7.21(b)所示。



(a)

i	v_i	最短路径P_i	路径长度
1	C	A→C	10
2	E	A→E	30
3	D	A→E→D	50
4	F	A→E→D→F	60
5	B	不存在	

(b)

图7.21 无向网G_6和从A到其余顶点的最短路径

(a) 无向网G_6； (b) 从顶点A到其余顶点的最短路径

7.5.1 单个源点到其余各顶点的最短路径（续）

与求最小生成树的普里姆算法类似，迪杰斯特拉算法也将图G的顶点集V分成两个子集U和$V-U$，U是已求得最短路径的顶点集。用一维数组Dist记录集合$V-U$中各顶点所求的当前最短路径信息。其定义如下：

```
typedef struct {
   int  prev;          // 当前最短路径上该顶点的前驱顶点的位序
   int  lowcost;        // 当前最短路径的长度
} DistInfo;            // V-U中顶点的当前最短路径信息
DistInfo Dist[MA_VEX_NUM];
int hpos[MAX_VEX_NUM];
```

借助Dist数组，迪杰斯特拉算法的步骤为：

（1）假设源点为v_i，初始：

U = { 源点v_i }　　V-U = { 除源点v_i之外的其它顶点 }

Dist[i] = {-1, 0}　　$v_i \in$U

Dist[j] = {i, w_{ij}}　　$v_i \in$V-U 且 $< v_i, v_j > \in$ E

Dist[j].lowcost = INFINITY　$v_i \in$V-U 且 $< v_i, v_j > \notin$E

（2）按路径长度升序，依次求得源点到其他所有顶点的最短路径，每次：

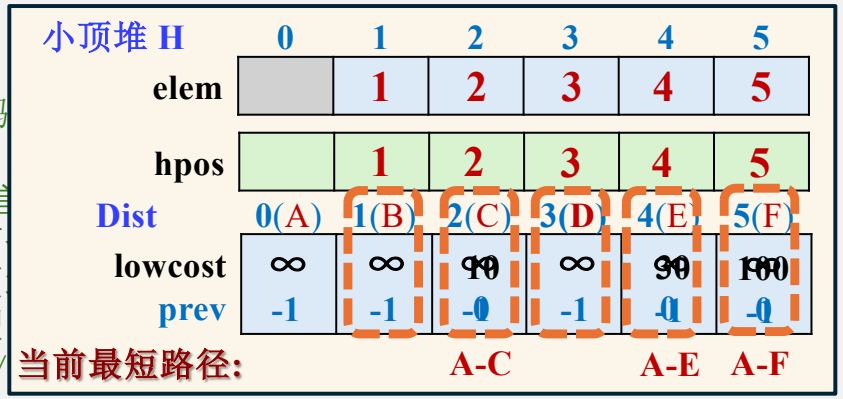
1）在Dist数组中按lowcost域找到最小值Dist[k]，求得从源点v_i到顶点v_k的最短路径，其长度为Dist[k].lowcost，将v_k加入到集合U，并从集合V-U删除。

2）更新Dist数组。检查经过已求得的最短路径P_k以及v_k到v_j（$v_j \in$ V-U）的弧，如果使v_j当前最短路径长度变短，则更新Dist数组的值。即若Dist[k].lowcost+W_{kj} < Dist[j].lowcost，则Dist[j] ={k, Dist[k].lowcost + W_{kj}}，否则不变。

7.5.1 单个源点到其余各顶点的最短路径（续）

以图7.21的无向网G_6为示例：

```
typedef struct {
    int  prev;          // 当前最短路径上该顶点的前驱
    int  lowcost;       // 当前最短路径的长度
} DistInfo;             // V-U中顶点的当前最短路径信
DistInfo Dist[MAX_VEX_NUM];   // 全局数组，当前
int hpos[MAX_VEX_NUM];        // 顶点位序在小顶
typedef int  HElemType;       // 小顶堆元素类型
Status lessPrior(HElemType x, HElemType y) /
    { return Dist[x].lowcost <= Dist[y].lowcost; }
```



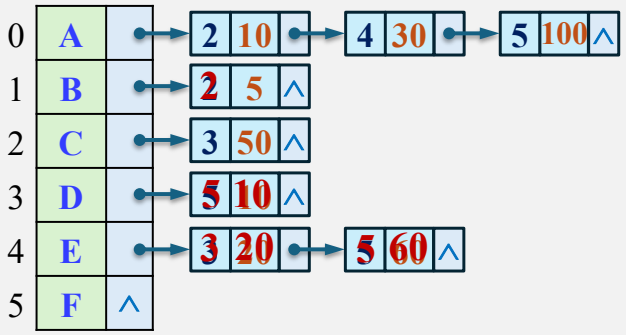
0 ——求A到其余顶点的最短路径

```
void ShortestPath_DIJ(ALGraph G, int i) {
   // 算法7.12  求有向网G中从i顶点到其他所有顶点的最短路径，并由全局Dist数组返回路径信息
   int j, m, k, min;    ArcNP p;
   Heap H = InitHeap(G->n);              // 初始化建容量为G的顶点个数的空小顶堆H
   for (j=0; j<G->n; j++)                // Dist初始化
      { Dist[j].prev = -1;    Dist[j].lowcost = INFINITY; }
   for (p=G->vexs[i].firstArc; p!=NULL;  p=p->nextArc) // 源点i引出弧信息存入Dist
      { Dist[p->adjvex].prev = i;    Dist[p->adjvex].lowcost = p->weight; }
   for (j=0; j<G->n; j++) if (j!=i) Insert(H, j); // 除源点外,各顶点序号插入最小堆H
   ..... /* 求最短路径 */
}
```

7.5.1 单个源点到其余各顶点的最短路径（续）

初始化之后的**当前状态**

开始求**0**顶点**A**到其余顶点的**最短路径**



小顶堆 H

	0	1	2	3	4	5
elem		3	4	3	1	5
hpos		4	1	3	2	5

K=

Dist	0(A)	1(B)	2(C)	3(D)	4(E)	5(F)
lowcost	0	∞	10	50	30	100
prev	-1	-1	0	4	0	4

当前最短路径: A-C A-E-D A-E A-E-D-F

…… /* 初始化 */

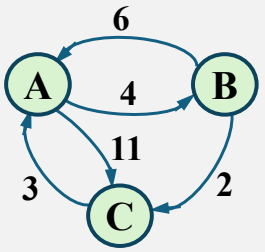
调用函数OutputPath,可由Dist数组信息输出从顶点A到第k顶点的最短路径

```
void OutputPath(ALGraph G, DistInfo Dist[], int k) {
    // 沿Dist数组的prev域，递归获得源点到k顶点的最短路径
    if (-1==k) return;                          // 上溯到路径尽头返回
    Outputpath(G, Dist, Dist[k].prev);          // 逆向递归获取路径上的顶点
    printf("%c ", G->vexs[k].data);             // 正向输出当前路径上的顶点
}
```

A C
A E D
A E
A E D F

}

7.5.2 每一对顶点之间的最短路径

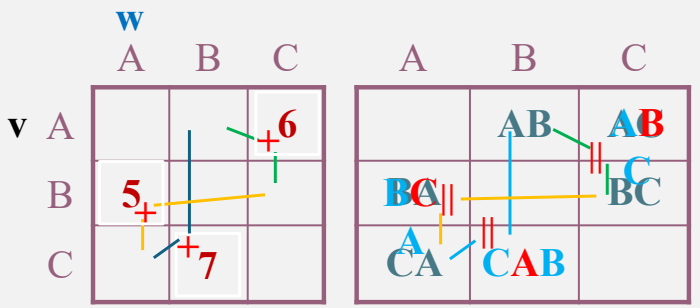


u				
0	A	0	4	11
1	B	6	0	2
2	C	3	∞	0

FLOYD算法先用三重循环将邻接矩阵复制到数组D，
初始化路径数组P，并登记顶点之间的每条边。

然后，用四重循环求各顶点之间的最短路径。

```
for( u=0;  u<G.vexnum; ++u )  // 考察每个u顶点
  for( v=0; v<G.vexnum; ++v ) // 相对于从v顶点 到 w
    for( w=0; w<G.vexnum; ++w ) // 的当前最短路径
      if( D[v][u] + D[u][w] < D[v][w] ) { // 经过u的路径是否更短？
        D[v][w] = D[v][u] + D[u][w];     // 更新当前最短路径长度
        for( i=0; i<G.vexnum; ++i)          // 更新各顶点之间的
          P[v][w][i] = P[v][u][i] || P[u][w][i]; // 当前最短路径
      }
```



最短路径长度数组D　　最短路径数组P

顶点到自身本无边，但约定有权值为0的虚拟“边”，即为到自身的最短路径。
因此，在算法穷举时，虽然会出现考察u、v和w中两者或三者相同的情况，
但由于是加“0”，故不影响求解结果。我们来看算法执行过程。

外三层循环一开始，u、v和w 均为0，即顶点A，条件语句不成立！
直到u=0(A)、v=1(B)和w=2(C)，
D[1][0] + D[0][2] < D[1][2] 即 6 + 11 < 2 的条件仍然不成立！

条件语句第三次成立出现在
u=0(A)、v=1(B)和w=2(C)，
此时，3 + 4 = 7 < 11，
即B经A到C的路径比直接B到C更短，
故 D[1][2]的11更新为7。
数组P中登记新当前最短路径 BAC

7.6 有向无环图及其应用

7.6.1 有向无环图

一个无环的有向图称做**有向无环图**(**directed acycline graph**)，简称**DAG图**。DAG图是一类较有向树更一般的特殊有向图，如图7.25列出了有向树、DAG图和有向图的例子。

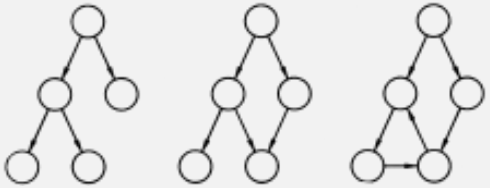
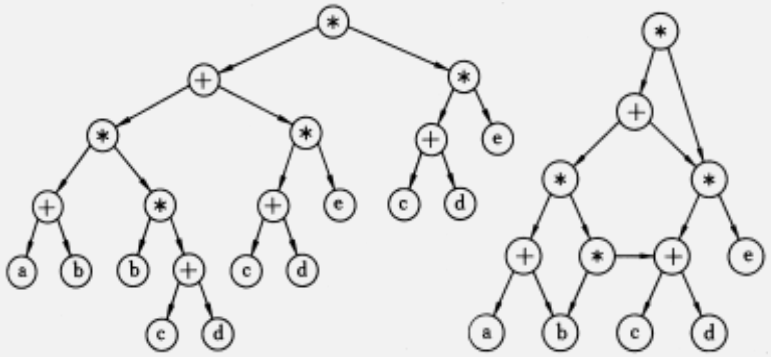


图7.25　有向树、DAG图和有向图示例

如图7.26所示，表达式

((a+b)*(b*(c+d))+(c+d)*e)*((c+d)*e)

可用二叉树或者有向无环图表示。



(a)　　(b)

图7.26　表达式的二叉树有向无环图描述示例

(a) 表达式的二叉树表示示例；　(b) 图(a)的二叉树的有向无环图表示

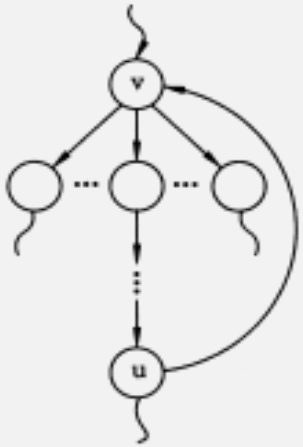


图7.27　含有环的有向图的深度优先生成树

检查有向图是否存在环比无向图复杂。

对于**无向图**，若深度优先遍历过程中遇到回边（即联接已访问过的顶点的边），则必定存在环；

对于**有向图**，这条回边有可能是指向深度优先生成森林中另一棵生成树上顶点的弧。

但如果从有向图某个顶点v出发的遍历，在DFS(v) 结束之前出现一条从顶点u到顶点v的回边（如图7.27所示），由于u在生成树上是v的子孙，则有向图中必定存在包含顶点v和u的环。

7.6.2 拓扑排序

拓扑排序（Topological Sort）是由某个集合上的一个**偏序**得到该集合上的一个**全序**。回顾离散数学中关于偏序和全序的定义：

若集合S上的关系R是自反的、反对称的和传递的，则称R是集合S上的偏序关系。

设R是集合S上的**偏序**（Partial Order），如果对任何$x, y \in S$必有xRy或yRz，则称R是集合S上的**全序**关系。

直观地看，**偏序**指集合中仅有部分成员之间可比较，而**全序**指集合中全体成员之间均可比较。例如，图7.28所示的两个有向图，图中弧(x, y)表示$x \le y$，则(a)表示偏序，(b)表示全序。若在(a)的有向图上人为地加一个表示$v_2 \le v_3$的弧（符号“≤”表示v_2领先于v_3），则(a)表示的亦为全序，且这个全序称为**拓扑有序**(Topological Order)，而由偏序定义得到拓扑有序的操作便是拓扑排序。

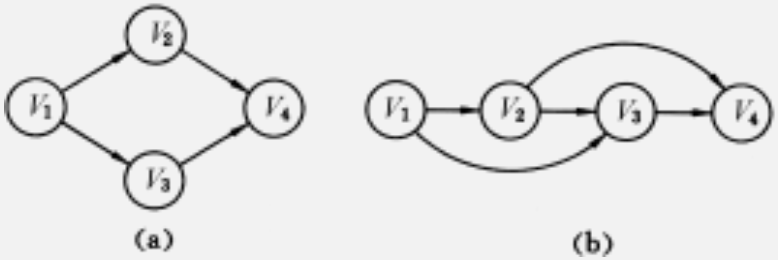


图7.28 表示偏序和全序的有向图
(a) 表示偏序；(b) 表示全序

表示偏序的有向图可用来表示一个流程图。它或者是一个施工流程图，或者是一个产品生产的流程图，再或是一个数据流图（每个顶点表示一个过程）。图中每一条有向边表示两个子工程之间的次序关系（领先关系）。

用**顶点表示活动**，用**弧表示活动间的优先关系**的有向图称为顶点表示活动的网(Activity On Vertex Network)，简称**AOV网**。在网中，若从顶点i到顶点j有一条有向路径，则i是j的前驱；j是i的后继。若$<i, j>$是网中一条弧，则i是j的直接前驱；j是i的直接后继。

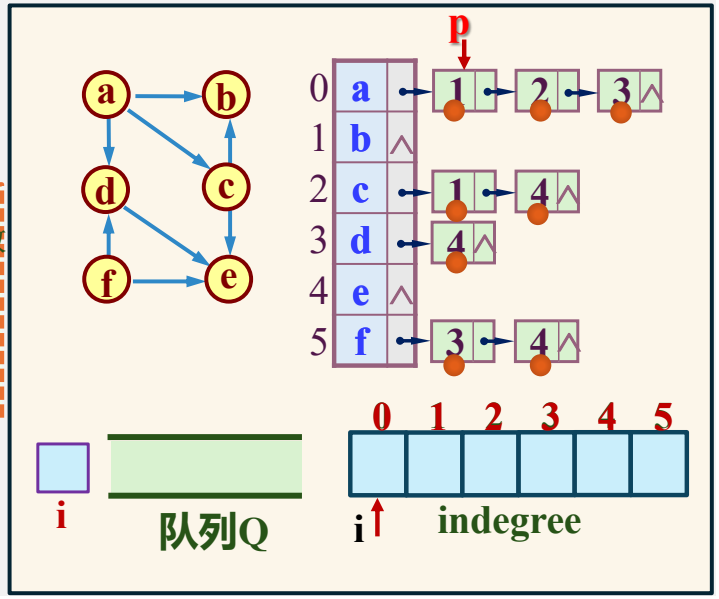
在AOV网中，不应该出现有向环，若存在环就意味着某项活动应以自己为先决条件。若设计出这样的流程图，工程便无法进行。而对程序的数据流图来说，则表明存在一个死循环。因此，对给定的AOV网应首先判定网中是否存在环。检测的办法是对有向图构造其顶点的拓扑有序序列，若网中所有顶点都在它的拓扑有序序列中，则不存在环。

7.6.2 拓扑排序（续）

对于任意一个有向图，其**拓扑排序过程**为：

（1）在有向图中任意选取一个入度为零的顶点，并输出；
（2）从图中删除该顶点及其所有引出弧；
（3）重复上述两步，直到图中不存在入度为零的顶点。此时，若图中所有顶点均已输出，则输出序列为拓扑序列，否则，图中存在回路，拓扑排序失败。

```
Status ToplogicalSort(ALGraph G) { // 算法7.14，对采用邻接表存储结构的图G进行拓扑排序
  ArcNP p;   int i, count = 0, *indegree;
  Queue Q = InitQueue();     // 初始化链队列Q
  if (!(indegree = (int*)calloc(G->n, sizeof(int)))) exit(OVERFLOW); // 自动清0
  for (i=0; i<G->n; i++)     // 计算每个顶点的入度
    for (p=G->vexs[i].firstArc; p!=NULL; p=p->nextArc)
      indegree[p->adjvex]++;
  for (i=0; i<G->n; i++)
    if (0==indegree[i]) EnQueue(Q, i); // 将入度为0的顶点入队
  while ((i=DeQueue(Q))!=nullE) {
    printf("%c ", G->vexs[i].data);  count++; // 输出顶点并计数
    for (p=G->vexs[i].firstArc; p!=NULL; p=p->nextArc)
      // 将i顶点的邻接顶点入度减1，若入度为0，则入队
      if (0 == --indegree[p->adjvex]) EnQueue(Q, p->adjvex);
  }
  free(indegree);                  // 回收入度数组
  if (count<G->n) return ERROR; // 若未输出所有顶点，存在回路
  else return OK;
}
```



7.6.3 关键路径

在一个有向网中，如果**顶点表示事件**，**弧表示活动**，弧上的权值表示活动持续的时间，则称该图为**边活动网**（Activity on Edge Network，简称**AOE网**）。

例如图9.32中的AOE网，包含11项活动：a_0，a_2，…，a_{10}，以及9个事件：v_0，v_1，…，v_8，其中事件v_3可理解为当活动a_2和a_4完成时，事件v_3发生，同时，当事件v_4发生之后，活动a_6才可以开始。

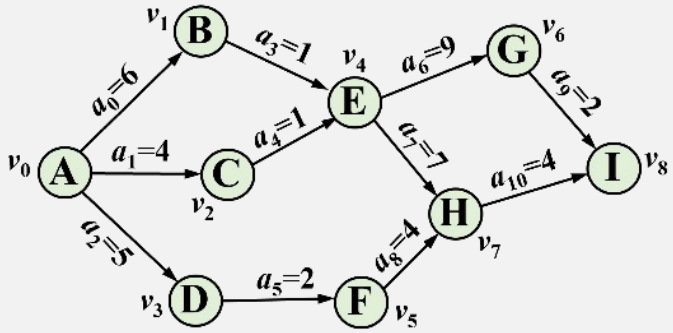


图9.32　AOE网及其关键路径示例

由于工程只有唯一一个开始点和结束点，故在正常情况下，称AOE网中唯一一个入度为零的顶点为**源点**，唯一一个出度为零的顶点为**汇点**。例如上图所示的AOE网，v_0是源点，v_5是汇点。

要估算完成整个工程所需的最短时间，就是要找一条从源点到汇点的最长路径（即权值之和最大的路径），该路径称为**关键路径**（Critical Path），关键路径上的活动称为**关键活动**，这些活动是影响整个工程工期进度的关键。

7.6.3 关键路径（续）

假设AOE网包含n个事件$v_0, v_1, \ldots, v_{n-1}$，和$m$个活动$a_1, a_2, \ldots, a_m$，$v_0$是源点，$v_{n-1}$是汇点。求关键路径的问题有以下四个相关概念：

（1）事件vi的**最早发生时间**$ve(i)$。$ve(i)$可定义成如下的递推公式：

$$ve(0)=0$$

$$ve(v_i)=\max_{k=1}^{m}\{ve(v_{j_k})+W_{j_k i}\} \quad 若<v_{j_k},v_i>\in E \quad 0<i\le n\text{-}1$$

（2）事件v_i的**最迟发生时间**$vl(i)$。$vl(i)$可定义为如下的递推公式：

$$vl(n-1)=ve(n-1)$$

$$vl(v_i)=\min_{k=1}^{m}\{vl(v_{j_k})-W_{ij_k}\} \quad 若<v_i,v_{j_k}>\in E \quad 0\le i<n-1$$

（3）活动a_k的**最早开始时间**$ae(k)$。活动a_k的最早开始时间即为vi事件的最早发生时间，即：

$$ae(k)=ve(i) \qquad 1\leqslant k\leqslant m$$

（4）活动ak的**最迟开始时间**$al(k)$。$al(k)$可定义为：

$$al(k)=vl(j)-W_{ij} \qquad <v_i,v_j>\in E \qquad 1\leqslant k\leqslant m$$

关键路径的计算过程是：**首先**，按拓扑排序过程，依次计算所有事件的最早发生时间，对事件v_i，更新其后继事件v_j的最早发生时间，若$ve(j)<ve(i)+W_{ij}$，则令$ve(j)=ve(i)+W_{ij}$，否则不变。例如，图7.34(a)所示的AOE网中顶点的最早发生时间计算过程如图7.35所示，指向顶点的最早发生时间的两个箭头表示$ve(i)$和W_{ij}相加。

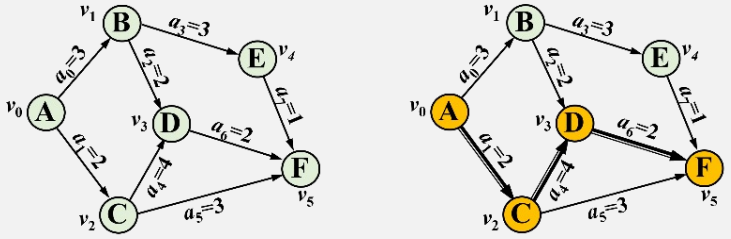


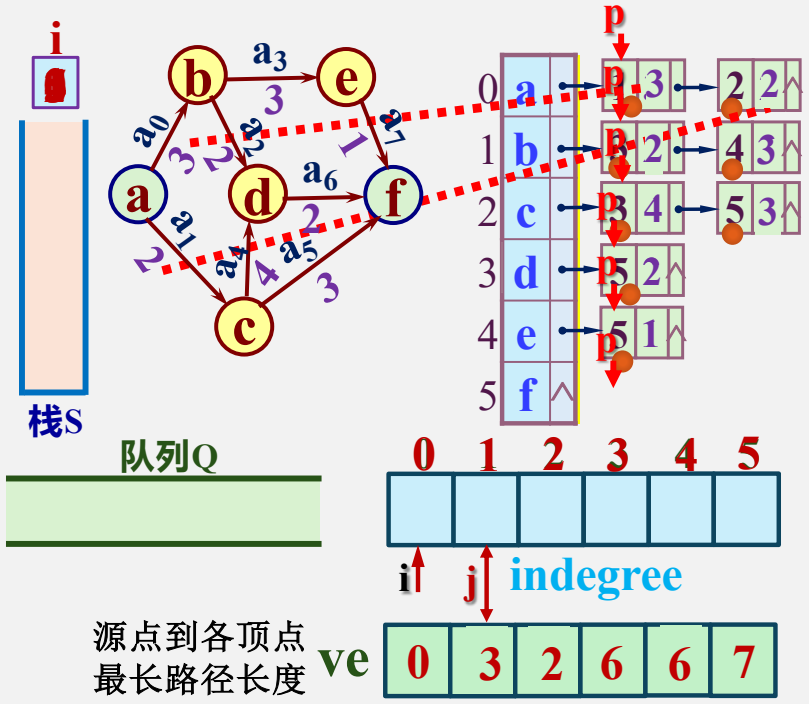
图7.34 AOE网及其关键路径示例

其次，按逆拓扑序列，依次计算所有事件的最迟发生时间，对事件v_i，更新其最迟发生时间，对于v_i的所有后继事件v_j，若$vl(i) > vl(j)-W_{ij}$，$vl(i)=vl(j)-W_{ij}$，否则不变。

求关键路径算法(一)

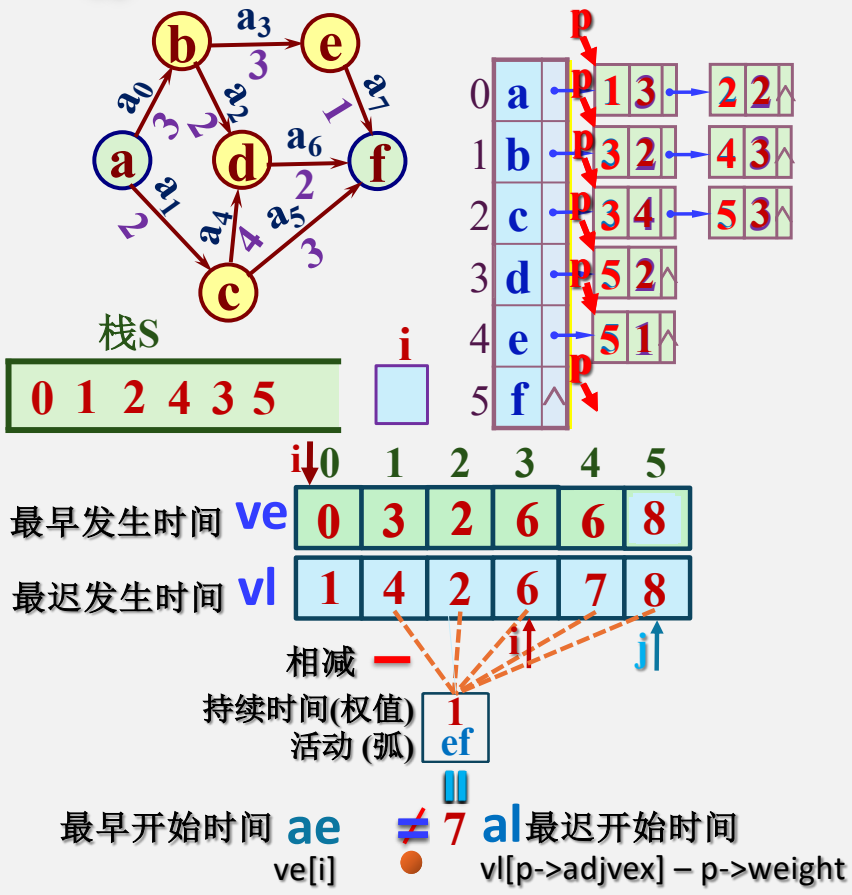
算法的主体是二重循环。只要Q队列不空，就队头元素出队到 i，并输出到 S栈。

内层for循环扫描第 i 顶点的邻接链表，对其指向的每个顶点的入度“减1”。




```
Stack TopologicalSort_ve(ALGraph G , int *ve) { // 算法7.15
 ArcNP p;   int I, j, *indegree;
 Stack S = InitStack(G->n, 5);  // 初始化顺序栈
 Queue Q = InitQueue();         // 初始化链队列
 if (!(indegree = (int*)calloc(G->n, sizeof(int)))) exit(OVERFLOW);
 for (i=0; i<G->n; i++)   // 计算每个顶点的入度
  for (p=G->vexs[i].firstArc;  p!=NULL;  p=p->nextArc)
    indegree[p->adjvex]++;
 for (i=0; i<G->n; i++) if (0==indegree[i]) EnQueue(Q, i);
 while ((i = DeQueue(Q)) != nullE) {
  Push( T, i);   // 顶点序号出 S 栈且入 T 栈
  for (p=G->vexs[i].firstArc; p!=NULL; p=p->nextArc)  {
   j = p->adjvex;   // 取i顶点序号
   if (0 == --indegree[     j     ]) EnQueue(Q, j);
   if (ve[i] + p->weight > ve[j]) ve[j]= ve[i]+p->weight;
  }      +   = 8 >
 }
 free(indegree);            // 回收入度数组
 if (StackLen(S)<G->n) return NULL; // 若未入栈所有顶点，则有回路
 else return S;
}
```

求关键路径算法(二)



下面结合示例演示算法7.16的执行过程。

有向网 G 及其邻接表如页面所示。

分配ve和vl空间后，首栈入栈算法7.15求 G 一个逆拓扑序列，并求得各顶点的最早发生时间ve。

算法中设一个域vl[i]保存了该弧的活动持续时间。

最迟发生时间vl各单元全置为汇点的ve值。

```
Status CriticalPath(ALGraph G) { // 求G中各项关键活动
  Stack S;  ArcNP p;  int  i, j, al, *ve, *vl;
  if (!(ve = (int*)calloc(G->n, sizeof(int)))) exit(OVERFLOW);
  if (!(vl = (int*)malloc(G->n*sizeof(int)))) exit(OVERFLOW);
  if ((S = TopologicalOrder(G, ve) == NULL)  return ERROR;
  for (i=0;  i<G->n;  i++) vl[i] = ve[G->n-1];   // vl[...]初值
  while ((i = Pop(S)) != nullE)  // 按拓扑逆序求各顶点的vl值
    for (p = G->vexs[i].firstArc;  NULL!=p;  p = p->nextArc) {
      j = p->adjvex;    //取邻接顶点(序号)
      if (vl[j] – p->weight < vl[i])  vl[i] = vl[j] - p->weight;  //刷新
    }
  for (i=0;  i<G->n;  i++)   // 求关键活动
    for (p=G->vexs[i].firstArc;  NULL!=p;  p = p->nextArc)
      if (ve[i] == vl[p->adjvex] – p->weight)
        p->flag = 1;  // 标记关键活动弧结点
  }
  free(ve);   free(vl);   return OK;
}
```